

Project Description

See Readme.md for run instructions. Text A search engine over public domain books from [Project Gutenberg](#), built using the [Gutendex API](#). Supports keyword search with AND logic and TF-IDF ranking, wildcard search via a permuterm index, and phrase search via a positional index across a corpus of downloaded books. The corpus is preprocessed through a standard NLP pipeline: tokenization, stopword removal, and Porter stemming via NLTK. Keyword results are scored using length-normalized TF-IDF, returning the top 15 results ranked by relevance. The system is exposed through a FastAPI backend and a React/Vite frontend with a search bar and per-result score display. All three indexes are serialized to disk with pickle and loaded once at server startup

AI AND LLM USAGE

As the landscape of software engineering is ever changing, so is the development of software. To keep up with the times, we have gone ahead and used AI assistance in developing this project. This will be explained in more detail as the AI usage pertains to each student.

I (Eduardo) used Claude and Claude code for two main purposes: 1. initial schema and code scaffolding, and 2. runtime and syntax error analysis. Like in many of our beginner computer science courses, having scaffolding streamlines implementation, like for this project. Thinking about how to exactly set up the project proved daunting at first, so having Claude handle that so I could do the development was far more efficient. I also used Claude, Claude Code specifically, to help me debug and implement the code when it was needed. This made the development process far more efficient, especially when I have little experience with the libraries and frameworks used to implement this application.

Andrew used Claude in the same way, but also to help do some of the tedious and menial tasks like writing up the initial version of this report. He used Claude Code for implementing the tf-idf scoring system as well.

The industry of software engineering follows this exact workflow, having an AI coding agent spit out initial code for a well described prompt, redesigning the prompts until it functions as intended. This is beneficial when it comes to saving time and resources, but can be detrimental if AI is not used properly or checked.

CORPUS

89 public-domain books successfully indexed from Project Gutenberg via the Gutendex API. The corpus covers a broad range of classic literature and non-fiction (e.g., Crime and Punishment, The Great Gatsby, Dracula, The Odyssey, War and Peace, Leviathan).

Three indexes are built during preprocessing:

- Inverted index (keyword search, TF-IDF ranked)
- Permuterm index (wildcard search)
- Positional index (phrase search)

All text is normalized via NLTK: tokenize → lowercase → remove stopwords → Porter stem.

SECTION 3: MEASURING PERFORMANCE

METRIC CHOICE

The system supports three query types with different retrieval behaviors, so different metrics apply to each. Keyword search returns results ranked by TF-IDF score, making rank-based metrics meaningful. Two metrics are reported:

- P@1 (Precision at 1): whether the top-ranked result is the correct book. This is the most practically important metric — a user who sees the right book first gets immediate value without scanning further.

- MRR (Mean Reciprocal Rank): the average of $1/\text{rank}$ of the first correct result across all queries. It captures partial credit when the correct book is retrieved but not at position 1.

Phrase and wildcard search return unordered sets — there is no principled ranking for these query types, as TF-IDF scores are defined per term and do not naturally extend to phrase proximity or wildcard expansion. For these, retrieval rate is reported: whether the correct book appears anywhere in the result set. NDCG is not used because it requires graded relevance judgments across multiple relevant documents per query, which would require substantial manual annotation effort.

TF-IDF FORMULATION

$\text{TF}(t, d) = \text{count}(t \text{ in } d) / \text{total_tokens}(d)$ (length-normalized)

$\text{IDF}(t) = \log((N + 1) / (df + 1))$ (smoothed)

$\text{score}(d) = \sum \text{TF}(t, d) \times \text{IDF}(t)$ for each query term t

Length normalization prevents longer books from being unfairly favored. Smoothing prevents division by zero and dampens terms that appear in every document.

TEST QUERIES

20 queries across all three query types, each with a verified correct book confirmed to be present in the index.

- Keyword (10): distinctive proper nouns and character names that appear almost exclusively in their source book.
- Phrase (5): two-word phrases. Three use adjacent content words; two contain a stopword between content words (marked [sw]), testing that the positional index handles this correctly.
- Wildcard (5): prefix patterns targeting unique character or title stems.

KEYWORD RESULTS (P@1 and MRR)

Query	Expected Book	Top Result	P@1	RR
raskolnikov	Crime and Punishment	Crime and Punishment	YES	1.00
gatsby	The Great Gatsby	The Great Gatsby	YES	1.00
karamazov	The Brothers Karamazov	The Brothers Karamazov	YES	1.00
zarathustra	Thus Spake Zarathustra	Thus Spake Zarathustra	YES	1.00
huckleberry	Huckleberry Finn	The Adventures of Tom Sawyer (rank 1)	NO	0.33
dracula	Dracula	Dracula	YES	1.00
beowulf	Beowulf	Beowulf	YES	1.00
odysseus	The Odyssey	The Odyssey	YES	1.00
leviathan	Leviathan	Leviathan	YES	1.00
gregor	Metamorphosis	Metamorphosis	YES	1.00

P@1: 0.900 (9/10 correct at rank 1)

MRR: 0.933

PHRASE + WILDCARD RESULTS (retrieval rate)

Query	Expected Book	Found	N
“dorian gray”	The Picture of Dorian Gray	YES	1
“sherlock holmes”	A Study in Scarlet (+Doyle)	YES	7

Query	Expected Book	Found	N
"great expectations"	Great Expectations	YES	6
"war and peace" [sw]	War and Peace	YES	10
"jekyll and hyde" [sw]	Dr. Jekyll and Mr. Hyde	YES	1
dracul*	Dracula	YES	1
karamaz*	The Brothers Karamazov	YES	2
zarathustr*	Thus Spake Zarathustra	YES	4
huckleberr*	Huckleberry Finn	YES	3
raskolnik*	Crime and Punishment	YES	1

Retrieval rate: 1.000 (10/10)

[sw] = phrase contains a stopword between content words

SECTION 4: ERROR ANALYSIS

OVERALL

Keyword search: P@1 = 0.900, MRR = 0.933. 9 of 10 queries ranked the correct book first; 1 query retrieved the correct book at rank 3. Phrase and wildcard retrieval: 10/10 correct.

ERROR CLASS 1: Ranking Failure — Shared Character, Length Bias

The one P@1 failure was "huckleberry":

Expected rank 1: Adventures of Huckleberry Finn (ID 76)

Actual rank 1: The Adventures of Tom Sawyer (ID 74)

Huckleberry Finn ended up ranked 3

Root cause: Huckleberry Finn is a prominent character in both books. Both contain the stemmed token "huckleberri" at similar raw frequencies, but Tom Sawyer is a shorter book. Because TF is normalized by document length, the same raw count produces a higher TF in a shorter document, giving Tom Sawyer a better TF-IDF score despite the query term naming the other book.

This is a fundamental limitation of bag-of-words TF-IDF: it has no understanding that a title match should outweigh a body mention, and length normalization can favor shorter documents when term frequency is similar. Approaches that could address this include field-weighted scoring (boosting title matches) or BM25 with a term-frequency saturation parameter.

ERROR CLASS 2: Low Precision on Broad Queries

Several queries retrieved the correct book but returned many unrelated results alongside it:

- "leviathan" → 7 results ("leviathan" appears as a sea-creature reference across texts)
- "war and peace" → 10 results (common English expression beyond the Tolstoy novel)
- "great expectations" → 6 results (common phrase in English)
- "sherlock holmes" → 7 results (Holmes referenced in passing across non-Doyle books)

For keyword queries, ranking mitigates this: the correct book tends to appear at rank 1 even when many books are returned. For phrase and wildcard queries there is no ranking, so users must scan all results manually.

SUMMARY

P@1 (keyword): 0.900 (9/10)
MRR (keyword): 0.933
Retrieval rate (phrase + wildcard): 1.000 (10/10)

Extra Credit Attempt: Field-Weighted Scoring

MOTIVATION

Error analysis of the baseline TF-IDF system identified one ranking failure: the query "huckleberry" ranked The Adventures of Tom Sawyer above Adventures of Huckleberry Finn. This happened because The Adventures of Tom Sawyer is a shorter book than The Adventures of Huckleberry Finn. We decided to try field-weighted scoring, specifically weighting titles to address this. This version of the project is stored in the .zip repo 'Field-Weighting.zip'

APPROACH: FIELD-WEIGHTED SCORING

The fix adds a title boost multiplier to the TF-IDF scoring function. During indexing, the title of each book is normalized (tokenized, lowercased, stopwords removed, Porter stemmed) and stored as a set of terms alongside the standard inverted index.

At query time, the scoring formula becomes:

$$\text{score}(d, q) = \sum \text{TF}(t, d) \times \text{IDF}(t) \times \text{boost}(t, d)$$

Where: $\text{boost}(t, d) = \text{TITLE_BOOST}$ if term t appears in doc d 's title, otherwise, the multiplier is set to 1.0

TITLE_BOOST is set to 3.0 — query terms that match a title contribute 3 times as much to the document's score as the same term appearing only in the body text. This was chosen arbitrarily.

RESULTS

Baseline system (TF-IDF, no title boost):

P@1: 0.900 (9/10 correct at rank 1)

MRR: 0.933

"huckleberry" → correct book at rank 3

After title boosting (TITLE_BOOST = 3.0):

P@1: 0.900 (9/10 correct at rank 1)

MRR: 0.950

"huckleberry" → correct book at rank 2

All other queries: unchanged (still rank 1 for all 9 that previously succeeded). No bugs introduced.

ANALYSIS

MRR improved from 0.933 to 0.950. The "huckleberry" query moved Huckleberry Finn from rank 3 to rank 2, confirming that the title boost is working in the right direction, albeit not enough to fully fix the ranking in this case. However, this is an edge case. While we understand that huckleberry should obviously return “The Adventures of Huckleberry Finn,” it does appear frequently in “The Adventures of Tom Sawyer.”

Despite not achieving a P@1 improvement on this specific query, the technique is sound and measurably improves MRR. A higher boost factor or true field-separated indexing could close the remaining gap.